# Lecture 1: Introduction to NLP

# Content

- **What is NLP?**
- **Overview of numerous applications of NLP in real-world scenarios**
- **Overview of heuristics, machine learning, and deep learning approaches**
- **Commonly used NLP pre-processing paradigms**

# What is NLP

- An area of computer science that deals with methods to analyze, model, and understand human language. Every intelligent application involving human language has some NLP behind it.

# Commonly used NLP pre-processing paradigms

- **Tokenization**: splitting large text samples into words / subwords

```
# Tokenization using nltk
from nltk.tokenize import word_tokenize
tokens = word_tokenize(text)
```

- **Regular Expressions**: Used for extracting, finding (or) replacing patterns in text

```
import re

# Extract all alphanumeric characters from text
tokens = re.findall("[\w']+", text)
```

- **Stemming**: Reduces words into their root words, using a rule based system. The root word is not necessarily a meaningful word.

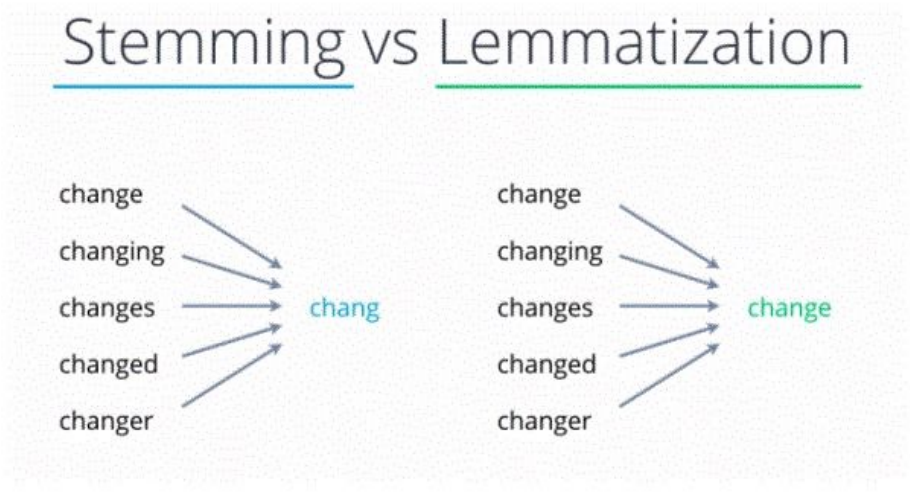
```
from nltk.stem PorterStemmer
stemmer = PorterStemmer()

stemmed_tokens = [stemmer.stem(token) for token in text]
```

- **Lemmatization**: Reduces words into their root words, using a dictionary. Makes sure that root words actually exist in english

```
from nltk.stem WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

lemmatized_tokens = [lemmatizer.lemmatize(token) for token in text]
```

- **Removing Stopwords**: Removing commonly occuring words like “a”, “an”, “the”, … which do not contribute much to semantic meaning of text

```
import nltk
nltk.download("stopwords")

stopwords_eng = stopwords.words('english')

filtered_tokens = [token for token in text if not token in stopwords_eng]
```




# Lecture 2 :Text Representation v1

# SpaCy

spaCy is a free, open-source library for advanced Natural Language processing (NLP) in Python. It's designed specifically for production use and helps you build applications that process and "understand" large volumes of text. Documentation: spacy.io

```
>>> $ pip install spacy
>>> import spacy
```

# Word Contractions

https://github.com/kootenpv/contractions: (https://github.com/kootenpv/contractions:) This package is capable of resolving contractions.

Usage

```
import contractions
contractions.fix("you're happy now")
# "you are happy now"
contractions.fix("yall're happy now", slang=False) # default: true
# "yall are happy"
contractions.fix("yall're happy now")
# "you all are happy now"
```

Adding Custom

```
import contractions
contractions.add('mychange', 'my change')
```

# Methods of Text Representation

- One-Hot
- Sparse
- Bag of Words
- TF-IDF

# One-Hot Encoding

In one hot encoding, every word (even symbols) which are part of the given text data are written in the form of vectors, constituting only of 1 and 0 . So one hot vector is a vector whose elements are only 1 and 0. Each word is written or encoded as one hot vector, with each one hot vector being **unique.**

| | Restaurant Reviews |
|---|---|
| R1 | Great restaurant and great service ! |
| R2 | They can do better to provide better service |
| R3 | Only two thumbs up, worst service ever |

Entire Corpus

Applying One-Hot:

| Set of all the words in the corpus | | R1: Great Restaurant and great service ! | R2: They can do better to provide better service | R3: Only two thumbs up, worst service ever |
|---|---|---|---|---|
| great | | 1 | 0 | 0 |
| restaurant | | 1 | 0 | 0 |
| and | | 1 | 0 | 0 |
| service | | 1 | 1 | 0 |
| they | | 0 | 1 | 0 |
| can | | 0 | 1 | 0 |
| do | | 0 | 1 | 0 |
| better | | 0 | 1 | 0 |
| to | | 0 | 1 | 0 |
| provide | | 0 | 1 | 0 |
| only | | 0 | 0 | 1 |
| Two | | 0 | 0 | 1 |
| thumbs | | 0 | 0 | 1 |
| up | | 0 | 0 | 1 |
| worst | | 0 | 0 | 1 |
| ever | | 0 | 0 | 1 |

# Sparse vectors

Optimizing One Hot Encoding using Sparse vectors.

Only storing indices of each word, to save space.

# Bag of Words

# Step 1: Determine the Vocabulary

Document:

- *the cat sat*
- *the cat sat in the hat*
- *the cat with the hat*

We first define our **vocabulary**, which is the set of all words found in our document set. The only words that are found in the 3 documents are: `the`, `cat`, `sat`, `in`, `the`, `hat`, and `with`.

# Step 2: Count

To vectorize our documents, all we have to do is **count how many times each word appears**:

| Document | the | cat | sat | in | hat | with |
|---|---|---|---|---|---|---|
| *the cat sat* | 1 | 1 | 1 | 0 | 0 | 0 |
| *the cat sat in the hat* | 2 | 1 | 1 | 1 | 1 | 0 |
| *the cat with the hat* | 2 | 1 | 0 | 0 | 1 | 1 |

# TF - IDF

Tf-Idf stands for **T**erm **f**requency-**I**nverse **d**ocument **f**requency. It tends to capture :

- How frequently a word/term ***Wi*** appears in a document ***dj*** **.** This expression can be mathematically represented by ***Tf(Wi, dj)***
- How **frequently** the same word/term appears across the entire corpus ***D***. This expression can be mathematically represented by ***df(Wi, D)*****.**
- ***Idf*** measures how **infrequently** the word ***Wi*** occurs in the corpus ***D.***

### Formula for Tf-Idf

$$X_{i,j} = tf(W_i, d_j).idf(W_i, D)$$

## Advantages

- Captures both the relevance and frequency of a word in a document.

## Drawback

- Each word is still captured in a standalone manner, thus the context in which it occurs is not captured.

# Method of Comparing words

# Cosine Similarity

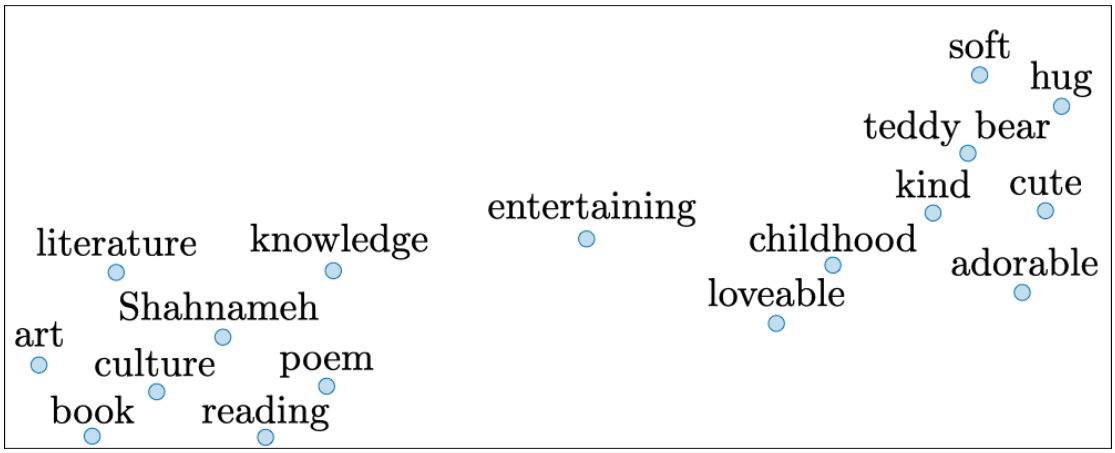
**Cosine similarity**

The cosine similarity between words A and *B* is expressed as follows:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}},$$

# T-SNE

*t*-SNE (*t*-distributed Stochastic Neighbor Embedding) is a technique aimed at reducing high-dimensional embeddings into a lower dimensional space. In practice, it is commonly used to visualize word vectors in the 2D space.

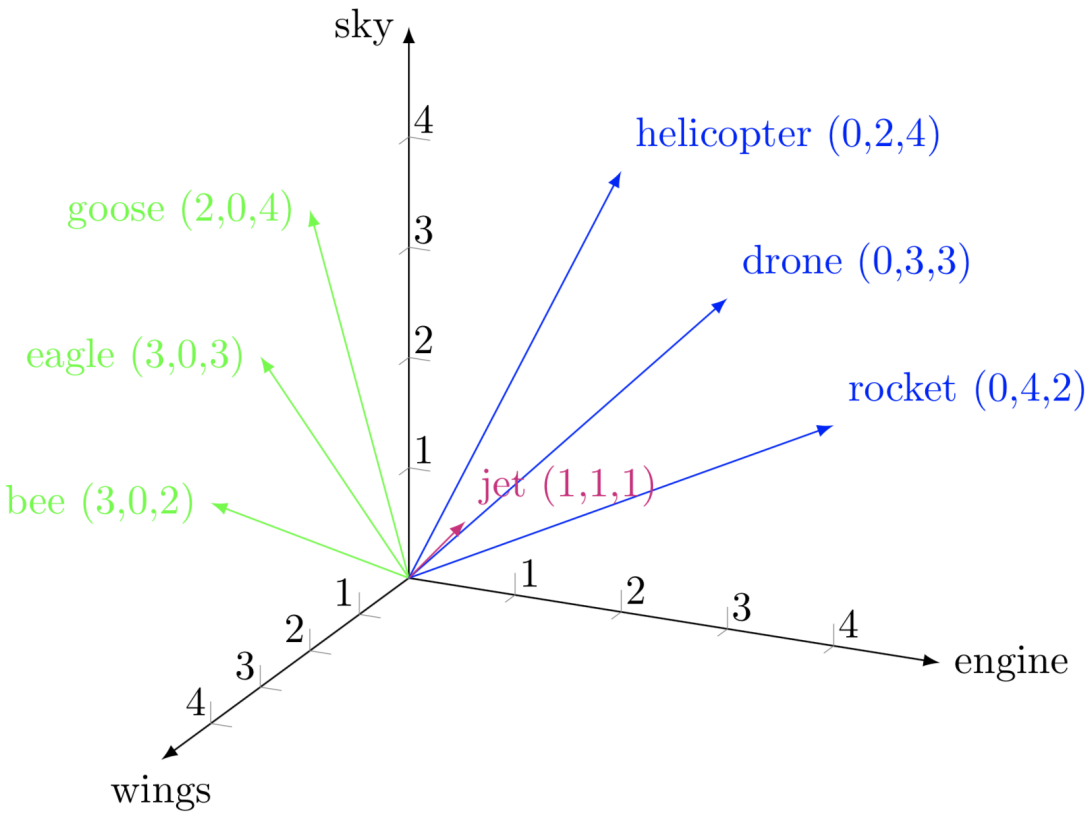



# Lecture 3: Word Embeddings v1

# Word Embeddings

A word embedding is **a learned representation for text where words that have the same meaning have a similar representation**

.




### Advantages over Discrete representations

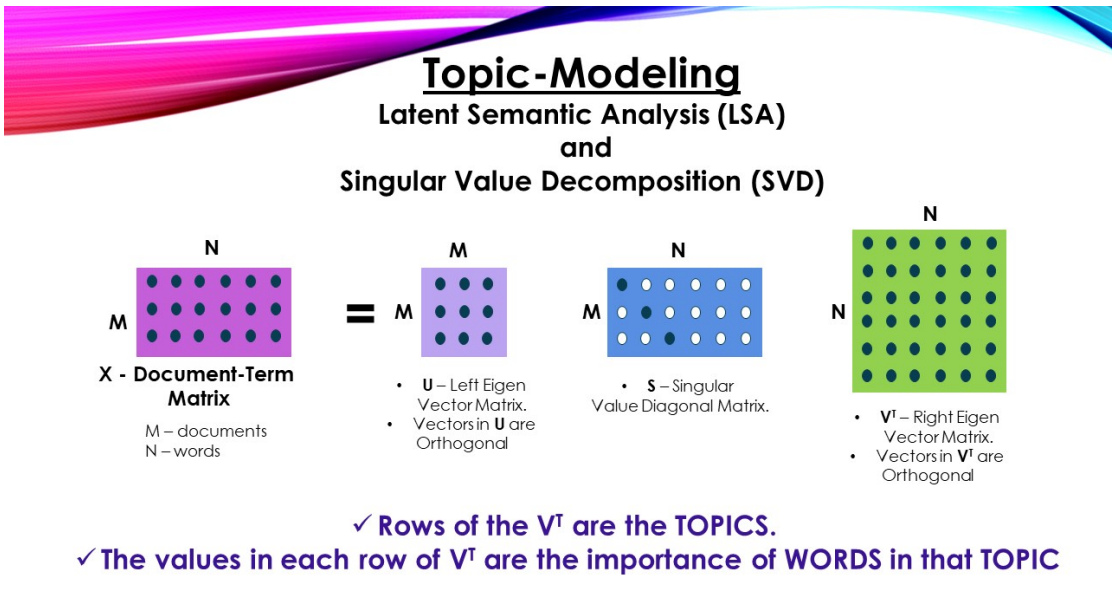
- Able to capture relations between different words.
- Ability to capture context by syntactic and semantic similarity

# Single Value Decomposition (SVD)

Let A be an m*n matrix. Then there exists a factorisation of A,

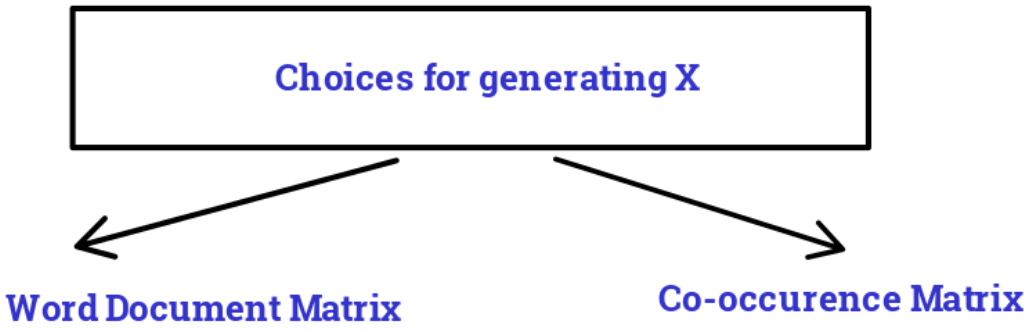
A = $U\Sigma V^T$

where **U** is an m**m orthogonal matrix, V is an n**n orthogonal matrix and **Σ** is an m*n matrix of the form,




**Steps:**

1. Looping over the corpus to **accumulate word co-occurence counts** in X matrix.
2. Perform **SVD on X** to get $U\Sigma V^T$ decomposition.
3. Use **rows of U as the word embeddings** for all words in our dictionary.




- **Assumption** (for both): Words that are related will often appear in the same documents and vice versa
- In 2nd choice, **X** contains word co-occurrences (i.e. affinity matrix)

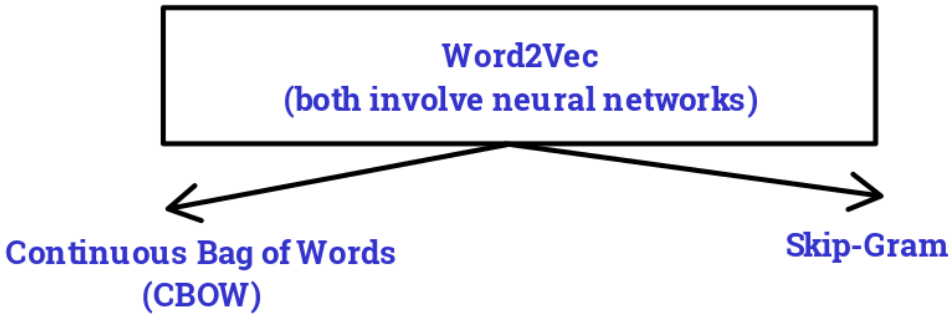
### Advantages and Disadvantages

| Advantages | Disadvantages |
|---|---|
| preserves the semantic relationship | poor scaling on large matrices (large memory). |
| efficient factorization | |
| computed only once, can be used multiple times. | |

# Word2Vec

The word2vec algorithm uses a neural network model to learn word associations from a large corpus of text.

**Methods of Word2Vec**



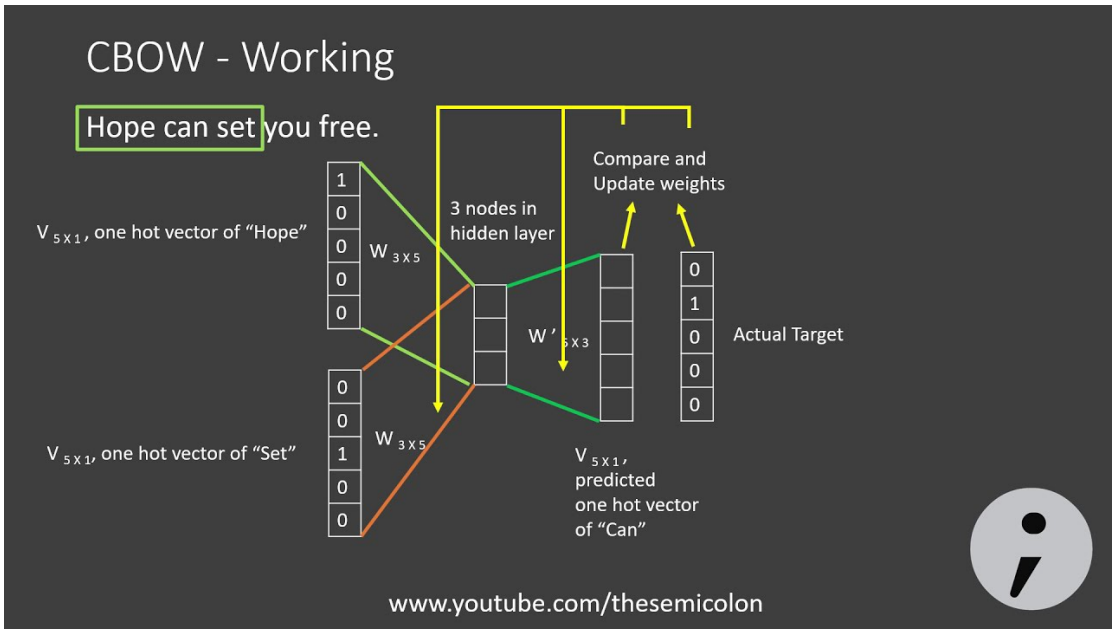

- **Continuous Bag of Words (CBOW):** predicts the current word from a window of surrounding context ****words.
- **Skip Gram:** the model **uses current word to predict** the surrounding window of **context words.**

## CBOW Architecture

| Input | Hidden Layer | Output |
|---|---|---|
| context words in a vector with size of vocabulary. | hyper-parameter which defines shape of word representations | outputs probability of all the words in a vector of size of the vocabulary. |

ReLU activation: Input to Hidden Layer

Softmax activation: Hidden to Output Layer




CBOW learns the word representations by reducing the **Cross-Entropy Loss (objective function)** during back-propagation

$$J = -\sum_{k=1}^{V} y_k \log \hat{y}_k$$

### Disadvantages of CBOW:

- Overfits on frequent words for context window > 1
- Doesn't produce good representations for rare words.

## Skip-Gram Method

Takes in **input** as **center/context word** and **predict** its **surrounding words**.

| Source Text | Training Samples generated from source text | | | |
|---|---|---|---|---|
| I will have orange juice and eggs for breakfast | (will, I) | (will, have) | (will, orange) | |
| I will have orange juice and eggs for breakfast | ( have, I) | (have, will) | (have, orange) | (have, juice) |
| I will have orange juice and eggs for breakfast | (orange, will) | (orange, have) | (orange, juice) | (orange, and) |
| I will have orange juice and eggs for breakfast | (juice, have) | (juice, orange) | (juice, and) | (juice, eggs) |
| I will have orange juice and eggs for breakfast | (and, orange) | (and, juice) | (and, eggs) | (and, for) |
| I will have orange juice and eggs for breakfast | (eggs, juice) | (eggs, and) | (eggs, for) | (eggs, breakfast) |
| I will have orange juice and eggs for breakfast | ( for, and) | ( for, eggs) | ( for, breakfast) | |

### Skip-Gram Architecture

| Input | Hidden Layer | Output |
|---|---|---|
| one-hot encoded representation | | |
| of the center word | Weighted sum of the input | |
| and weight matrix WVxN | dot product | |
| between the output from hidden layer h1 and $W'$NxV | | |

Skip-gram learns word representations via:

1. matching output and true probabilities
2. **Log Likelihood** objective function.

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c, j\neq 0}\log p(w_{t+j}|w_t)$$

# Lecture 4: Language Modeling

# Content

- Probabilistic language models
- N-gram models
- Maximum Likelihood estimation
- Evaluation
  - Average Log-likelihood
  - Cross entropy

    - Perplexity
- Accounting OOV (Out of vocabulary words)
    - Laplace smoothening
        - K smoothening
        - Add-1 smoothening
    - Backoff & Interpolation
- Ambiguitiy & Limitations

# Probabilistic Language Models

Probabilistic Language Models learn probability of next token, based on previous tokens. Given that a set of words $w_1, w_2, w_3, w_4, \ldots, w_k$, predict the next word, $w_{k+1}$ as $P(w_{k+1}|w_k, w_{k-1}, w_{k-3}, \ldots)$. This is multi-class classification problem inspired by naive-bayes

# N-gram models

An N-gram language model predicts the **probability of a given N-gram within any sequence of words** in the language.

Consider "I have a dream" exists within a corpus.
$P(I\ have\ a\ dream) = P(I) \cdot p(have) \cdot p(a) \cdot p(dream)$

Eg: a tri-gram model:

| < start > | I | want | a | dream | job | have | dog | company |
|---|---|---|---|---|---|---|---|---|
| (< start >, I) | 0 | 0 | 0.0714 | 0 | 0 | 0 | 0.142 | 0 |
| (I,want) | 0 | 0 | 0 | 0.0714 | 0 | 0 | 0 | 0 |
| (want,a) | 0 | 0 | 0 | 0 | 0.0714 | 0 | 0 | 0 |
| (a,dream) | 0 | 0 | 0 | 0 | 0 | 0.0714 | 0 | 0 |
| (dream,job) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (i,have) | 0 | 0 | 0 | 0.142 | 0 | 0 | 0 | 0 |
| (have,a) | 0 | 0 | 0 | 0 | 0.0714 | 0.0714 | 0 | 0 |
| (a,dog) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (dream,company) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (company, < / start>) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (job, < / start>) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| (dog, < / start>) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

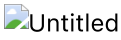
# Building N-grams: Maximum Likelihood Estimation

Maximum likelihood estimation maximizes a likelihood function, that, under the assumed statistical model, the observed data is most probable.

For n-grams, likelihood function for most probable word at position $k$ is
$P(w_k|w_{k-1}, w_{k-2}\dots w_1) = \frac{Count(w_k, w_{k-1})}{Count(w_{k-1}, w_{k-2}\dots w_1)}$

N-gram models are trained by building a dictionary that given a set of $n-1$ words, predicts $n^{th}$ word by maximum likelihood estimation. Thus, n-gram assumes that

**N-gram assumption: Probability of each word is independent, given the words before it and only depends on fraction of their occurance within the corpus**




# Evaluation

In order to evaluate the performance of n-gram (or) autoregressive models, following metrics are used

- ***Average Loglikelihood:*** of a text document from a corpus is
  - $\log P_{eval}(text)_{avg} = \frac{1}{||word||} \sum_{word} \log P_{train}(word)$
  - where $||word||$ is number of words in the text
- ***Cross entropy****:* of a text document from a corpus is negative of average log likelihood
  - Cross Entropy (text) = $-\log P_{eval}(text)_{avg}$
- ***Perplexity***: of a text document from a corpus is defined as exponential of cross entropy
  - $Perplexity(text) = e^{cross\_entropy(text)}$
  - A lower perplexity indicates higher models' understanding of a language

# Accounting OOV (Out of vocabulary words)

In order to account for OOV words, following algorithms are used

- Laplace smoothening helps to tackle problem of zero probability. We add an additional `[UNK]` token to training set and modify likelihood function
  - **K-Smoothening**:

$$P(w_k|w_{k-1}, w_{k-2}\dots w_1) = \frac{Count(w_k, w_{k-1}) + k}{Count(w_{k-1}, w_{k-2}\dots w_1) + k \cdot V}$$

    - where $k$ is a randomly chosen hyperparameter and $V$is vocabulary size (Number of unique n-grams in the training corpus)
  - **Add 1 Smoothening**:

$$P(w_k|w_{k-1}, w_{k-2}\dots w_1) = \frac{Count(w_k, w_{k-1}) + 1}{Count(w_{k-1}, w_{k-2}\dots w_1) + V}$$

    - where $V$is vocabulary size (Number of unique n-grams in the training corpus). Add 1 smoothening is a simplified version of K-Smoothening where $K = 1$
  - Effect of Smoothening

$$P_{train}([UNK]) = \frac{k}{V}$$

- Back-off and Interpolation
  - **Back-off**: this technique falls back to lower-order N-gram models when a higher-order N-gram model is unavailable

$$P_{backoff}(w_n|w_{n-1}, w_{n-2}, \dots w_1) = \begin{cases} P_{train}(w_n|w_{n-1}, w_{n-2}, \dots w_1) & Count(w_n, w_{n-1}, \dots w_1) \geq 0 \\ P_{backoff}(w_{n-1}|w_{n-2}, w_{n-3}, \dots w_1) & otherwise \end{cases}$$

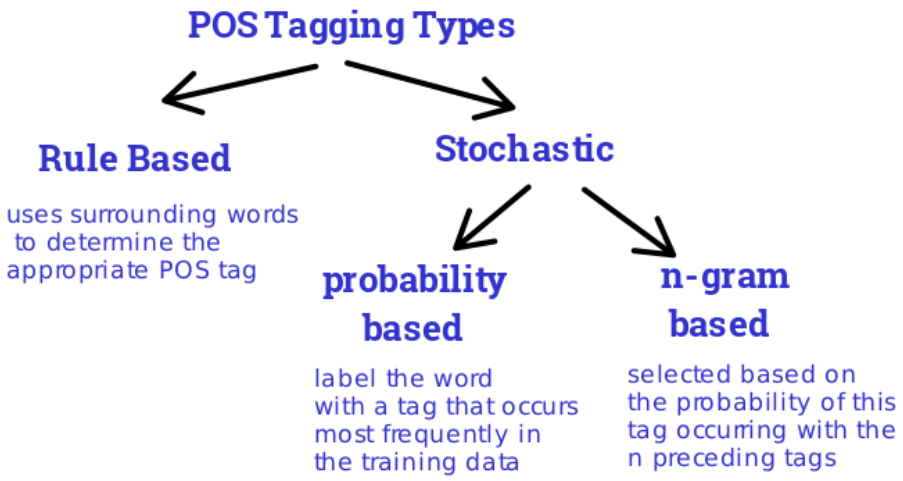
- **Interpolation**: This technique combines likelihoods of multiple N-gram models to estimate next word. In linear interpolation, we typically take weighted sum of likelihoods as a predictor for next token

$$P(w_n|w_{n-1}, w_{n-2}, \dots w_1) = \lambda_1 P(w_1) + \lambda_2 P(w_2|w_1) + \dots \lambda_{n-1} P(w_{n-1}|w_{n-2}, w_{n-3} \dots w_1) + \lambda_n P(w_n|$$

# Lecture 5: POS Tagging and Topic Modelling

# Part of Speech (POS) Tagging

process of classifying and labelling words into appropriate parts of speech, such as noun, verb, adjective, adverb, conjunction, pronoun and other categories.

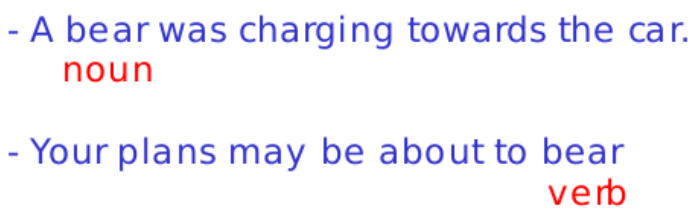



Examples of POS tags in NLTK (library of python):

- VBG – verb, present participle or gerund,
- PRP – pronoun, personal,
- NN – noun, common, singular or mass

### Some Applications of POS Tagging

- Text to speech conversion
- Disambiguation of statements (check the word **bear**):

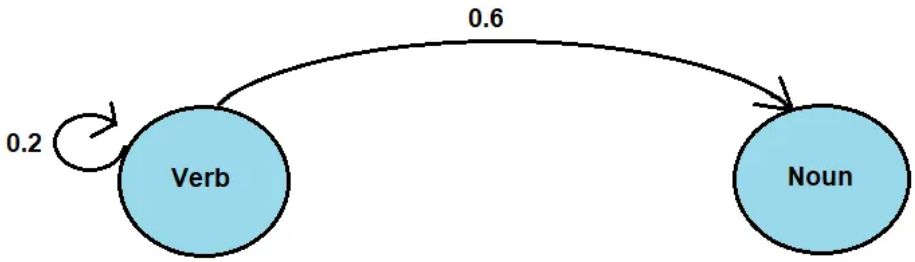



- Named Entity Recognition, etc.

## Markov Chains

**Markov Model:** representation of states and transitions to different states by assuming future states depend only on current state

E.g.




Current state → tail of the arrow

Future state → head of the arrow

number on the arrow → Likelihood of tail followed by head

## Transition Matrix

Another way to represent transition probabilities

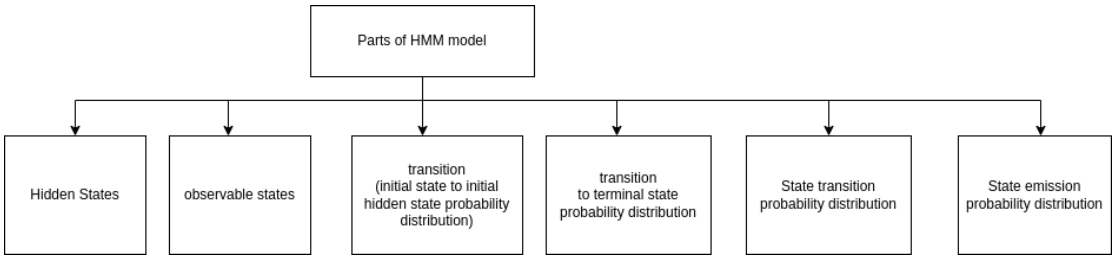
| | NN | VB | O |
|---|---|---|---|
| (initial) | 1/3 | 0 | 2/3 |
| NN (noun) | 0 | 0 | 1 |
| VB (verb) | 0 | 0 | 0 |
| O (other) | 6/14 | 0 | 8/14 |

**Corpus:**
<s> in a station of the metro

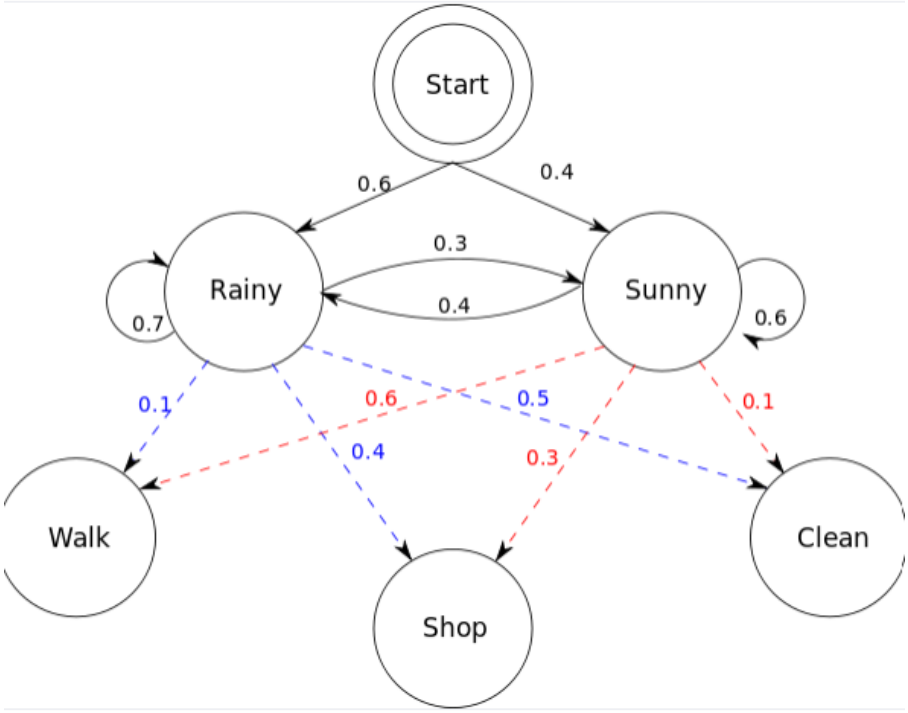
<s> the apparition of these faces in the crowd :

<s> petals on a wet, black bough .

## Hidden Markov Model (HMM)




**Example illustration**



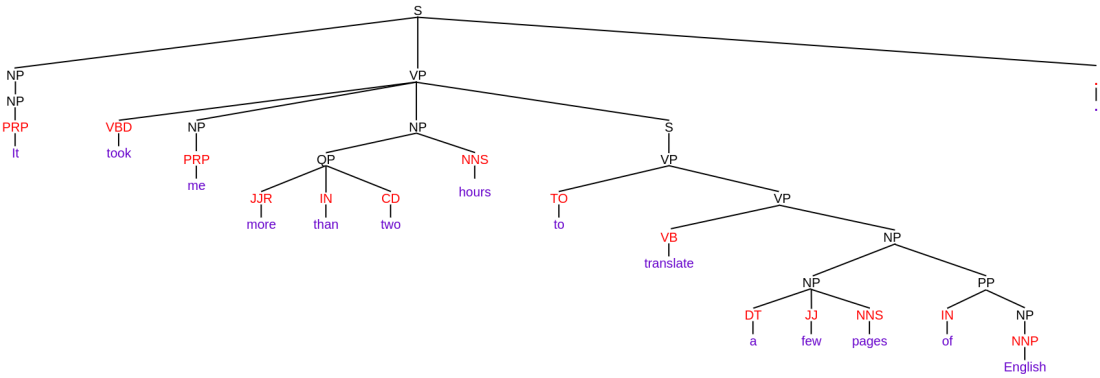

Dotted lines → transition to visible states

Observable (visible) states → words of the sentences

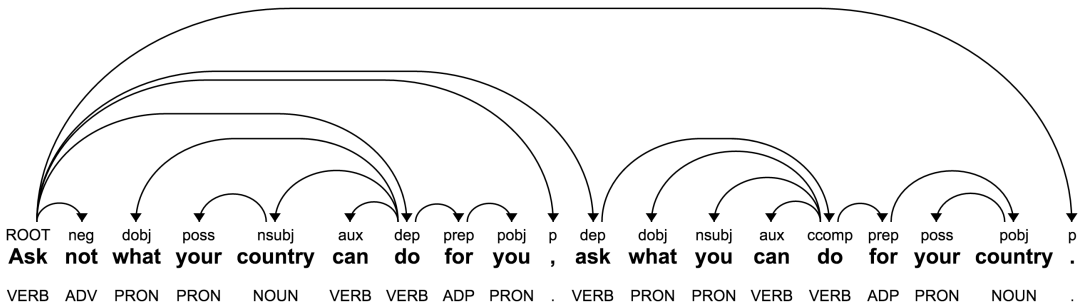
Hidden states → Parts of Speech

## Constituency Parsing

Process of analysing the sentences by breaking down it into sub-phrases also known as constituents




## Dependency Trees

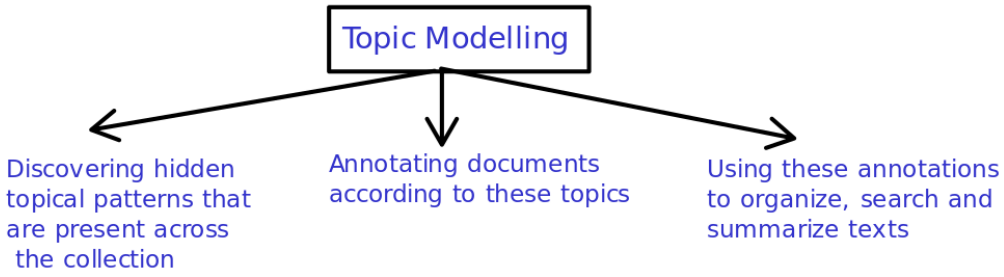



Arrows → parent-child relationship

Root → word with no arrow pointing towards it

# Topic Modelling

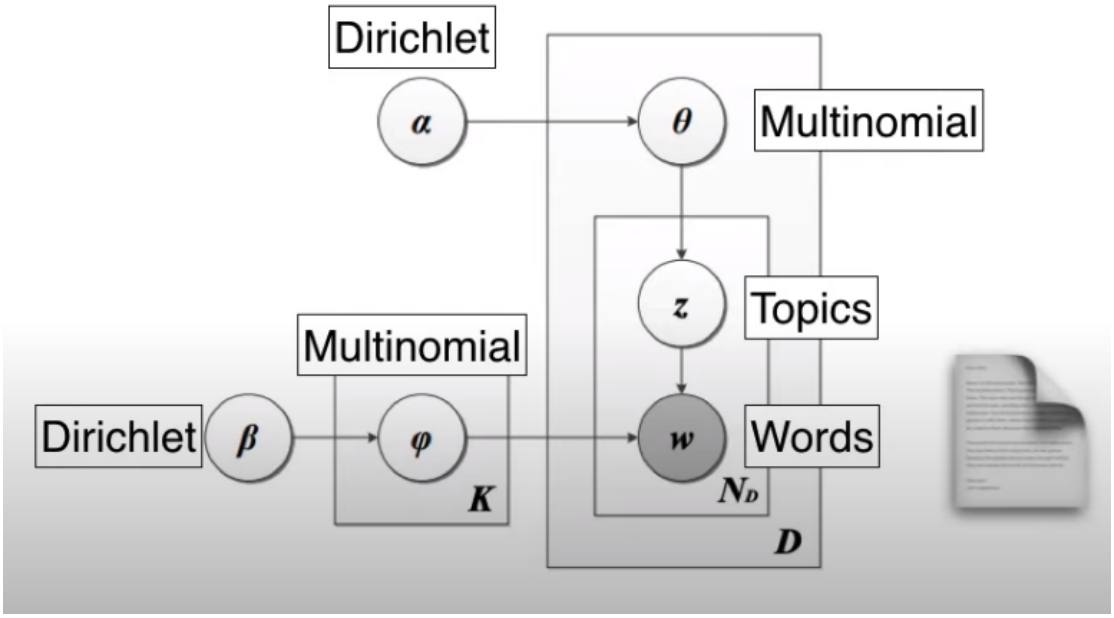
**Topics:** Representative group of words in a large corpus.




### Latent Dirichlet Allocation (LDA)

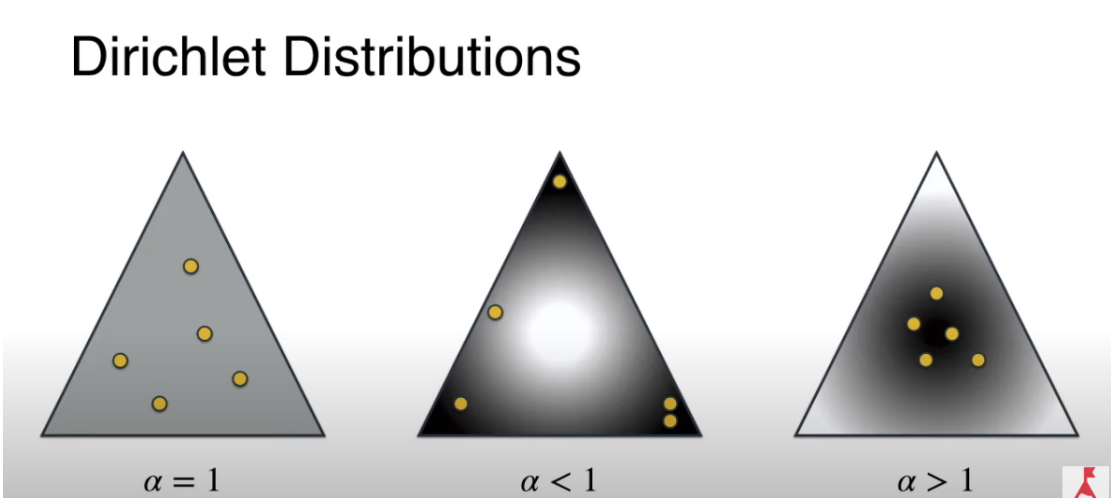
- LDA assumes that documents are composed of words that help determine the topics and maps documents to a list of topics by assigning each word in the document to different topics.
- While identifying the topics in the documents, it starts with random assignment of topics to each word and iteratively improves the assignment of topics to words through ***Gibbs sampling.***

### Architecture of LDA:




### LDA Hyperparameters

| Hyper-parameter | usage |
|---|---|
| 'α' | document-topic density factor |
| 'β' | topic-word density factor |
| 'K' | number of topics to be considered (predefined) |




Corners → Topics

Dots → Articles

Middle distribution represents the distribution of articles w.r.t. topics (as articles generally belong to a unique topic, lower probability of belonging to 2, and so on)

References:

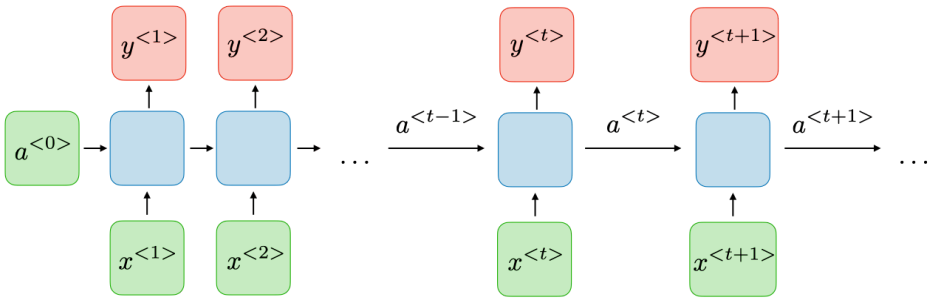
Latent Dirichlet Allocation (Part 1 of 2) (https://www.youtube.com/watch?v=T05t-SqKArY)

Understanding Latent Dirichlet Allocation (LDA) (https://www.mygreatlearning.com/blog/understanding-latent-dirichlet-allocation/#latent-dirichlet-allocation-lda)

# Lecture 6: Vanilla RNN

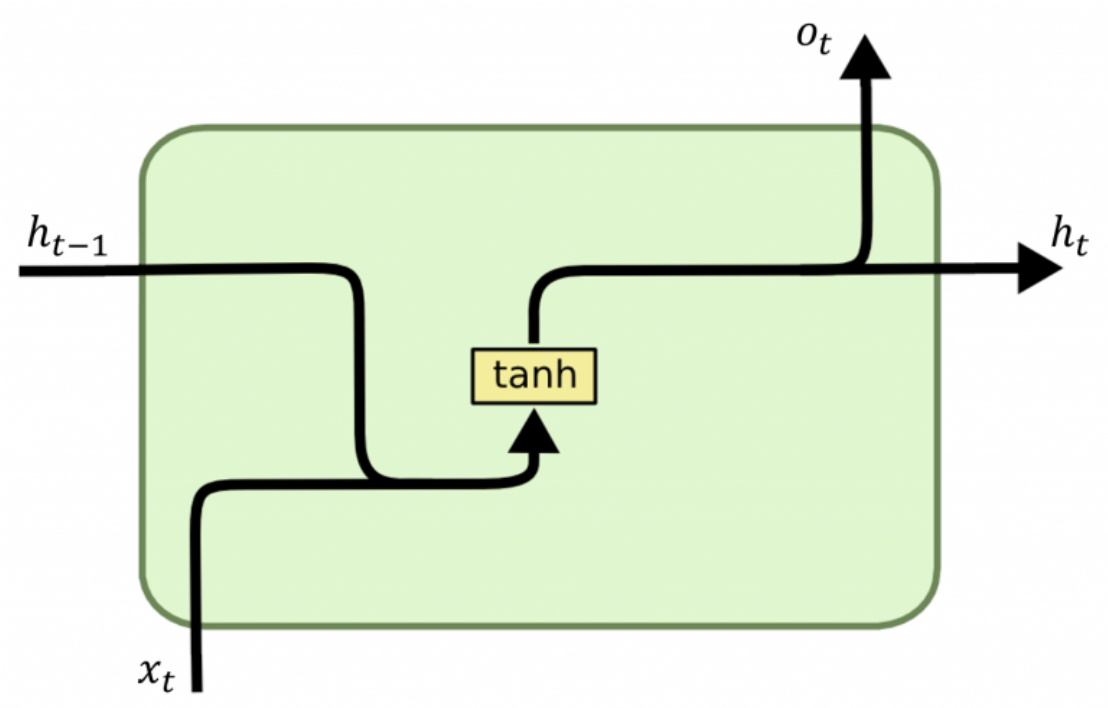
## RNN

Recurrent neural networks, also known as RNNs, are a class of neural networks that allow previous outputs to be used as inputs while having hidden states. They are typically as follows:



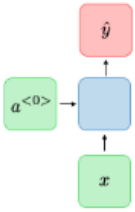
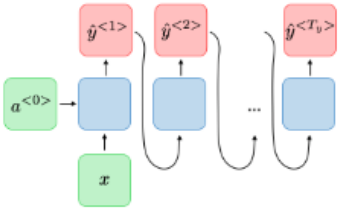
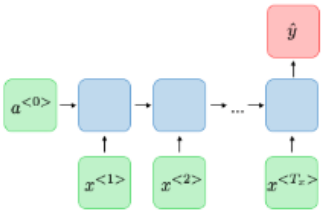
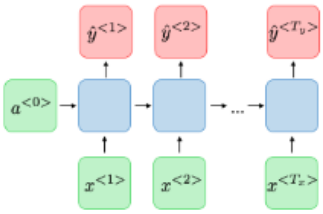
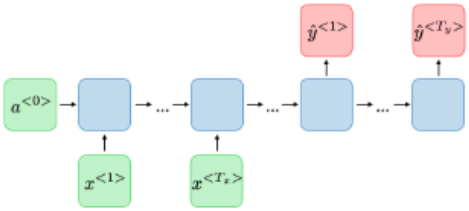

For each timestep *t* , the input x^t the activation a^t and the output y^t .

### What are different components of the RNN unit?



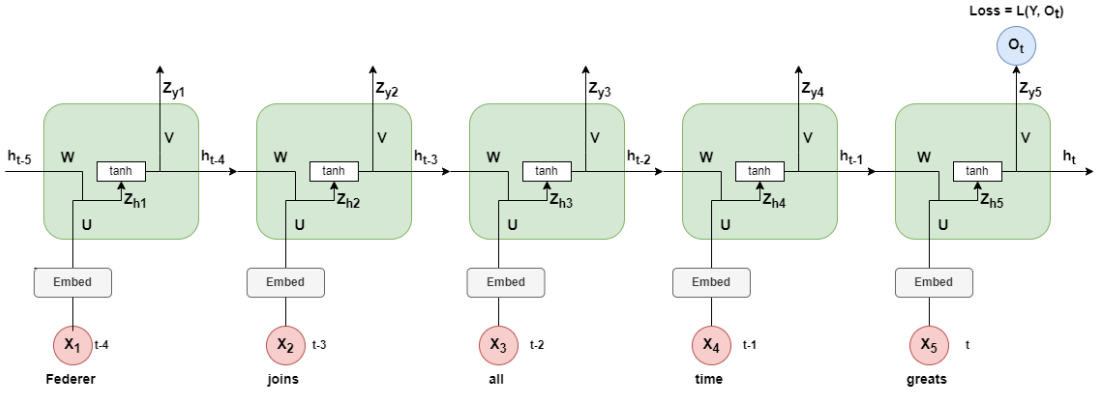

- **Xt**: Xt is the input at time step t.
- **ht-1**: Hidden state from the previous time step which is vector representation of the information from previous time steps.
- **tanh**: The input from the current time step **t** . In case of Vanilla RNN it is the **tanh.**
- **ht**: Hiddent state from the current time step **t**.
- **Ot**: Ot is the output of RNN.

## Types of RNN

| Type of RNN | Illustration | Example |
|---|---|---|
| One-to-one $T_x = T_y = 1$ |  | Traditional neural network |
| One-to-many $T_x = 1, T_y > 1$ |  | Music generation |
| Many-to-one $T_x > 1, T_y = 1$ |  | Sentiment classification |
| Many-to-many $T_x = T_y$ |  | Name entity recognition |
| Many-to-many $T_x \neq T_y$ |  | Machine translation |

# Propagation in RNN

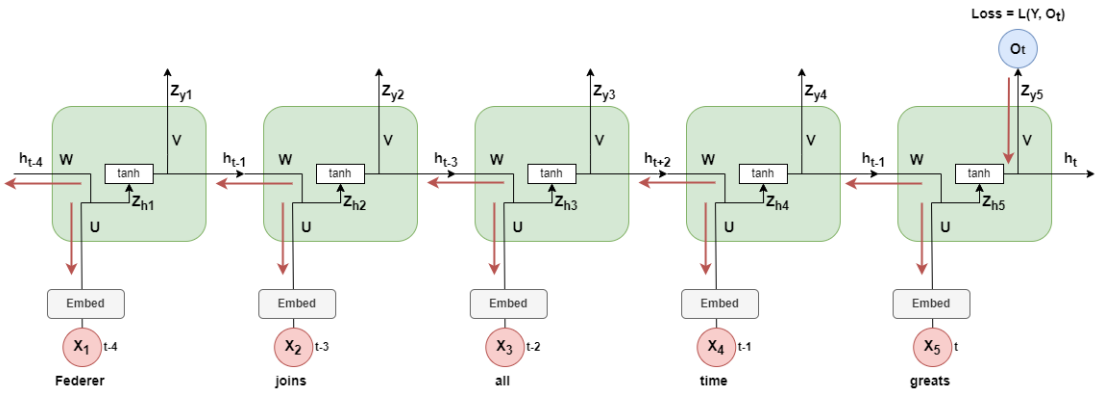
## Forward Propagation:




- **Whx = U**: The weight matrix at the input
- **Whh = V**: The weight matrix at the hidden state
- **Why = W**: The weight matrix at the output
- **Zyt, Zht =** Intermediate results

- **σ =** Activation function
- **Y =** Actual
- **L =** Loss function

### Back Propagation:




### Steps involved in updating parameter weights:

- Calculate the gradients of the loss with respect to the parameters
- Multiply it with the Learning rate
- Update the new weights

# Loss function:

In the case of a recurrent neural network, the loss function ***L*** of all time steps is defined based on the loss at every time step as follows:

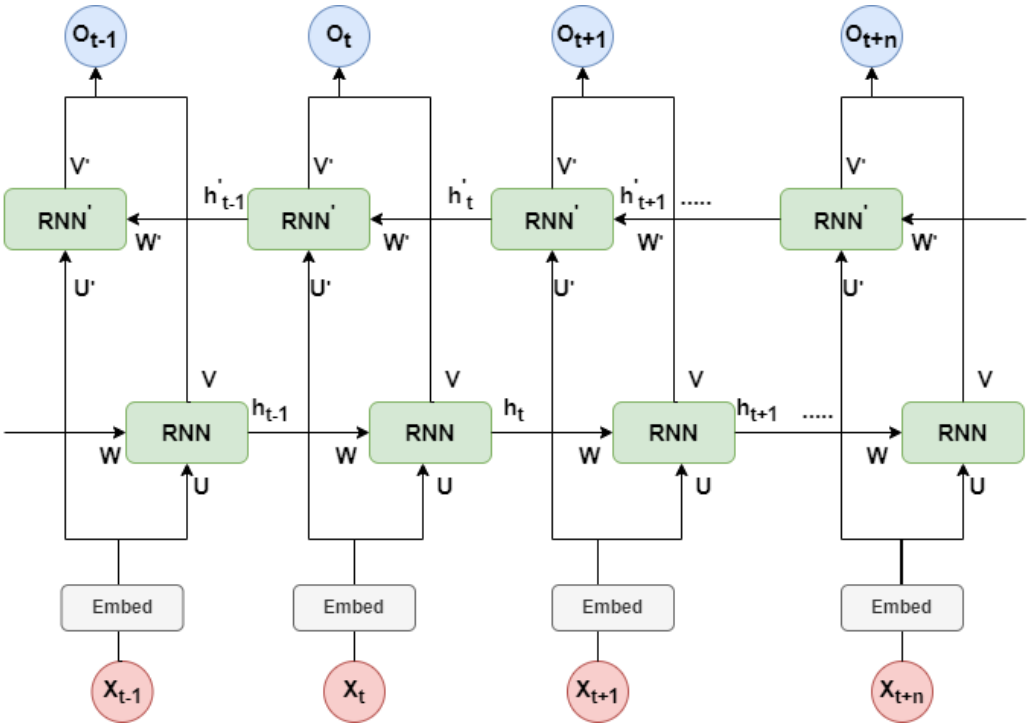
$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

# Back propagation through time:

Backpropagation is done at each point in time. At timestep ***T*** , the derivative of the loss ***L*** with respect to weight matrix ***W*** is expressed as follows:

$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(T)}}{\partial W}\Big|_{(t)}$$

# Bidirectional RNN




# Lecture 7: LSTMs

# Content

- RNNs and Long term dependencies
- Truncated backpropagation through time
- Long Short Term Memory Networks
  - LSTM Architecture
    - Forget gate
    - Input gate
    - Output gate
  - Solution to Vanishing & Exploding gradients
- LSTM Varients: GRU

# RNNs on Long Term Dependencies

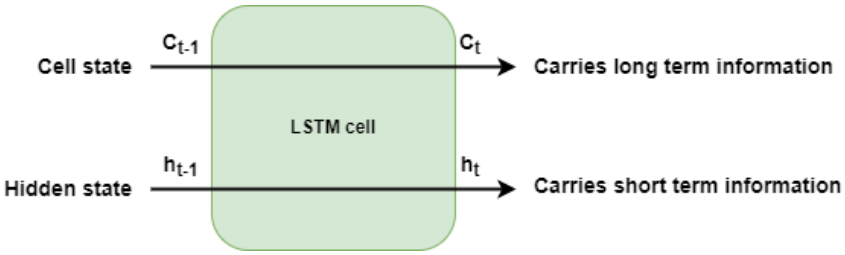
**Long term dependencies** occur in text when understanding context of a token requires us to refer to tokens farther away in text. RNNs cannot capture these long term dependencies from final encoder hidden state due to vanishing & exploding gradients

# Truncated backpropagation through time

In truncated backpropagation through time, input sequence is split into chunks of specified window and backpropagation is performed on them. This results in faster training but long term dependencies are not learnt if they're not in the same window.

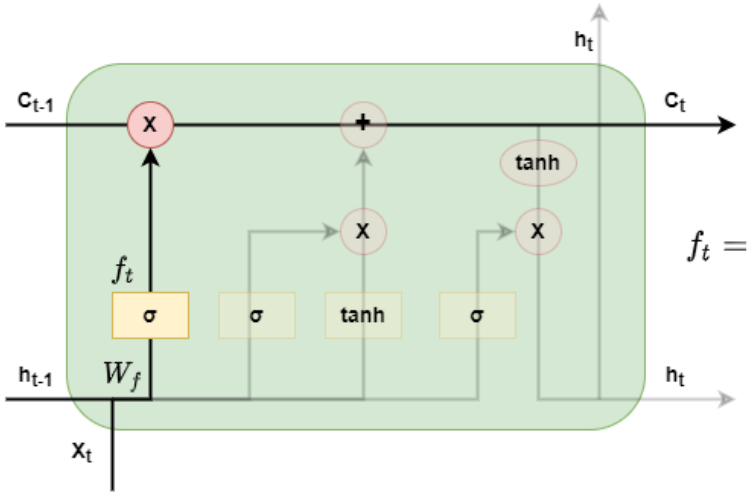
# Long Short Term Memory Networks

LSTM units carry information across different LSTM cells by **hidden state** and **cell state.**



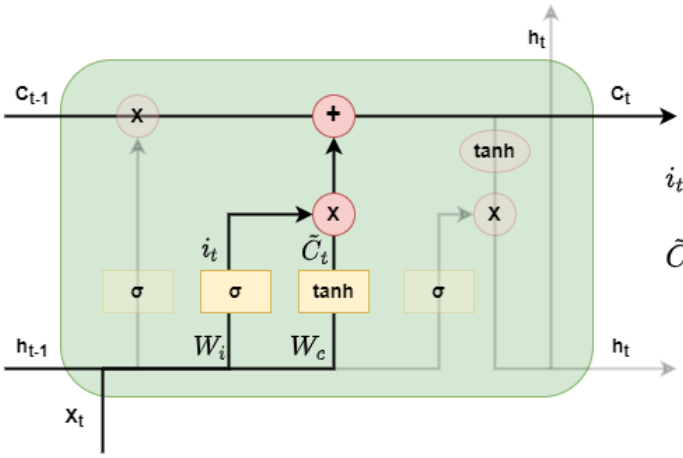

Propation within an LSTM cell can be broken down into 3 parts

- Forget gate: responsible for deciding what information to forget in cell state, based on current input and hidden states.




$$f_t = \sigma(W_f * [h_{t-1}, x_t] + b_f)$$

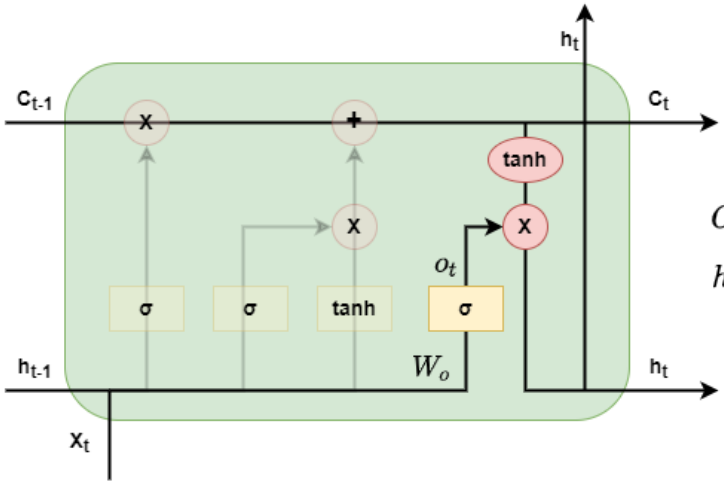
- Input gate: decides what information to add to the cell state, from the input and hidden states.




$$i_t = \sigma(W_i * [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = tanh(W_C * [h_{t-1}, x_t] + b_c)$$

- Output gate: decides what relevant information has to be passed from cell state to the next hidden state




$$O_t = \sigma(W_o * [h_{t-1}, x_t] + b_o)$$

$$h_t = O_t * tanh(C_t)$$

- Solution to vanishing & exploding gradients problem: unlike RNNs, gradients of LSTM do not have a fixed pattern and can take any positive value at any time step, thus mitigating Vanishing and Exploding gradients problem

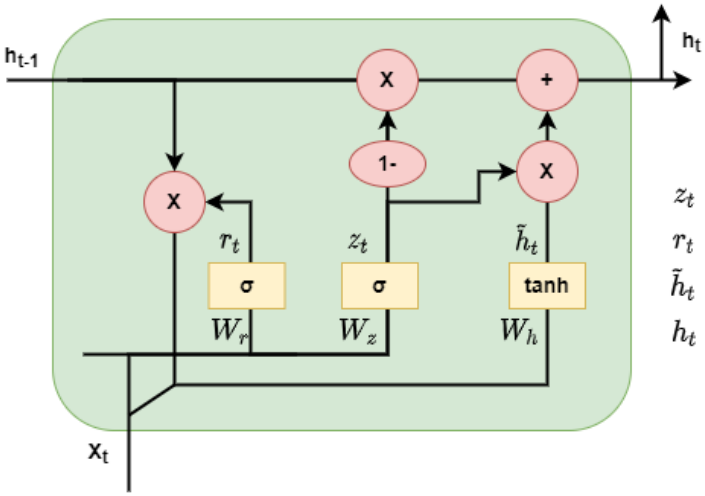
- For example, consider gradients of cell state:

$$\frac{\partial \mathbf{C_t}}{\partial \mathbf{C_{t-1}}} = \frac{\partial \mathbf{C_t}}{\partial \mathbf{f_t}} \frac{\partial \mathbf{f_t}}{\partial \mathbf{h_{t-1}}} \frac{\partial \mathbf{h_{t-1}}}{\partial \mathbf{C_{t-1}}} + \frac{\partial \mathbf{C_t}}{\partial \mathbf{i_t}} \frac{\partial \mathbf{i_t}}{\partial \mathbf{h_{t-1}}} \frac{\partial \mathbf{h_{t-1}}}{\partial \mathbf{C_{t-1}}} + \frac{\partial \mathbf{C_t}}{\partial \tilde{\mathbf{C}}_\mathbf{t}} \frac{\partial \tilde{\mathbf{C}}_\mathbf{t}}{\partial \mathbf{h_{t-1}}} \frac{\partial \mathbf{h_{t-1}}}{\partial \mathbf{C_{t-1}}} + \frac{\partial \mathbf{C_t}}{\partial \mathbf{C_{t-1}}}$$

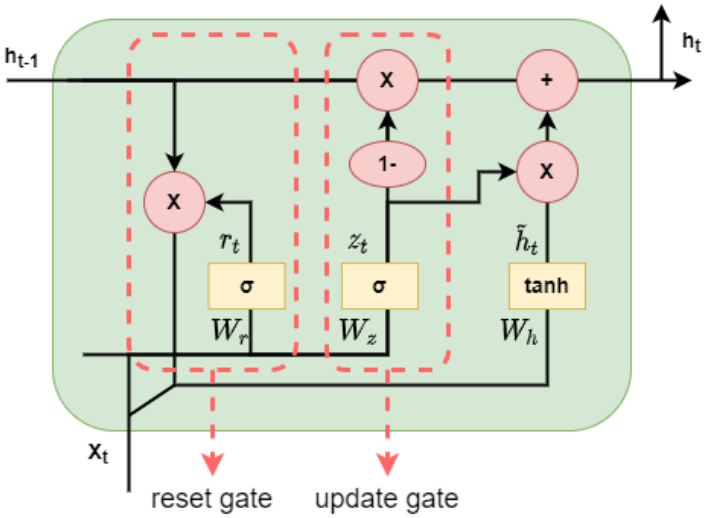
# LSTM Varients: GRU

GRU architecture has two gates and a single hidden state

- Reset gate: responsible for deciding which information from hidden state to remove
- Update gate: responsible for deciding which information from inputs to add to hidden state




$$z_t = \sigma(W_z * [h_{t-1}, x_t])$$
$$r_t = \sigma(W_r * [h_{t-1}, x_t])$$
$$\tilde{h}_t = tanh(W_h * [h_{t-1}, x_t])$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

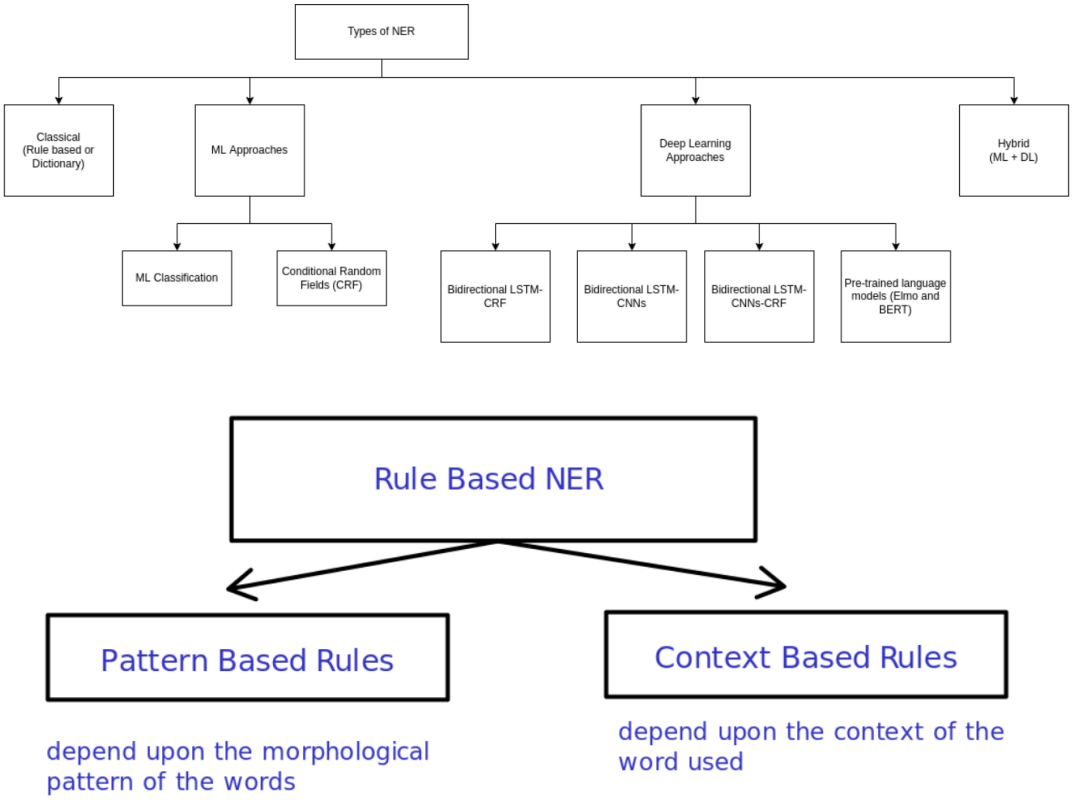



# Named Entity Recognition

**Named entity recognition (NER)** is a subtask of information retrieval concerned with the automatic extraction of named mentions of entities, where the set of possible entity types originally consisted of people, organizations, and locations

| Type | Tag | Sample Categories | Example sentences |
|---|---|---|---|
| People | PER | people, characters | **Turing** is a giant of computer science. |
| Organization | ORG | companies, sports teams | The **IPCC** warned about the cyclone. |
| Location | LOC | regions, mountains, seas | The **Mt. Sanitas** loop is in **Sunshine Canyon**. |
| Geo-Political Entity | GPE | countries, states, provinces | **Palo Alto** is raising the fees for parking. |
| Facility | FAC | bridges, buildings, airports | Consider the **Golden Gate Bridge**. |
| Vehicles | VEH | planes, trains, automobiles | It was a classic **Ford Falcon**. |

## Ways to do NER




## Sequence Labelling

Assignment of labels/tags to each element of a sequence being passed as an input using an algorithm or machine learning model.

Used for handling:

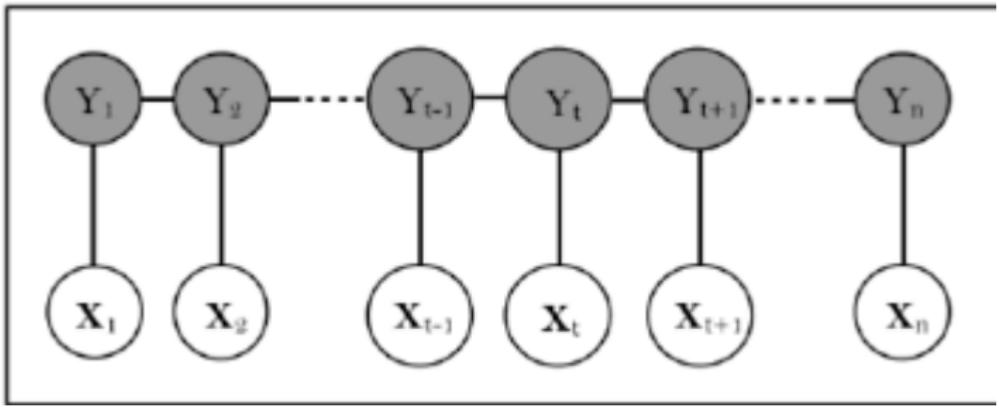
- Segmentation(Detection) ambiguity
- Tag assignment(Recognition) ambiguity

## Conditional Random Fields

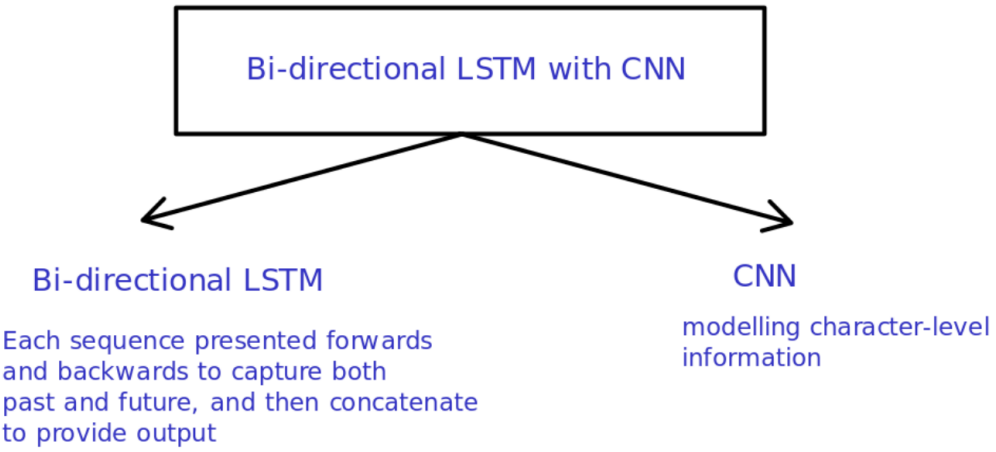
- They are a type of probabilistic graph model that take neighboring sample context into account for tasks like classification.
- Prediction is modeled as a graphical model, which implements dependencies between the predictions.

## Linear CRF

A linear chain CRF confers to a labeller in which tag assignment(for present word, denoted as $y_i$) depends only on the tag of just one previous word(denoted by $y_{i-1}$)




### Deep Learning Approaches




### Bi-directional LSTM with CRF

CRF helps in Sequence Labelling and considering correlations between labels in neighbourhoods.

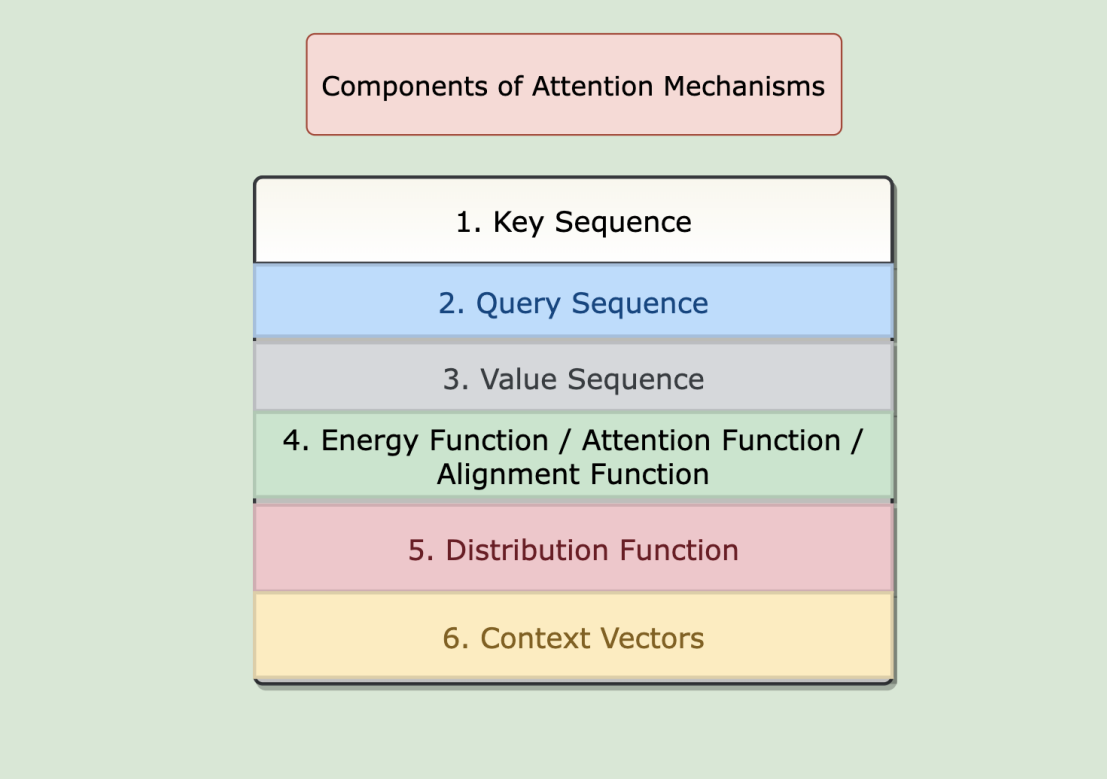
# Lecture 9: Attention

### What is Attention

### *Attention mechanisms Consist of following Componets:*

Attention is proposed as a solution to the limitation of the Encoder-Decoder model which encodes the input sequence to one fixed length vector from which to decode the output at each time step.

1. `Key(K)`
2. `Query(Q)`
3. `Value(V)`
4. `Energy Function(a)`
5. `Alignment distribution(g)`
6. `Final Representation(C)`




### Formula for Attention

$$attention\_score = Attention\_function(Q,\ K)$$
$$attention\_distribution = softmax(attention\_score)$$
$$Context\_vectors = attention\_distribution \times V$$

where,

$$K = Key\ vector$$
$$Q = Query\ vector$$
$$V = Value\ vector$$
$$attention\_score = energy\_score$$
$$attention\_distribution = attention\_weights$$

*For Our Task,*
*K = LSTM(Input Sequence)*
*Q = Embedding(aspect_term)*
*V = K*

### 1. Key

K encodes the data features whereupon attention is computed.

### 2. Query

The attention mechanism will give emphasis to the input elements relevant to the task according to Q.Q is represented by embeddings of actual textual queries, contextual information,etc.

### 3. Attention Function/ Energy function

From the Keys and Query, a vector E of n energy scores/attention score e_i is computed through a alignment function α

$E=\alpha(Q,K)$

### 4. Alignment Distribution

Attention scores are then transformed into attention weights using distribution function g

$W=g(E)$

Such weights are the outcome of the core attention mechanism.The commonest distribution function is the softmax.

### 5. Value

Many tasks require the computation of new representation of the keys. In such cases, it is common to have another input element; a sequence V of n vectors vi , the values, representing the data where upon the attention computed from K and Q is to be applied.

### 6. Final Representation.

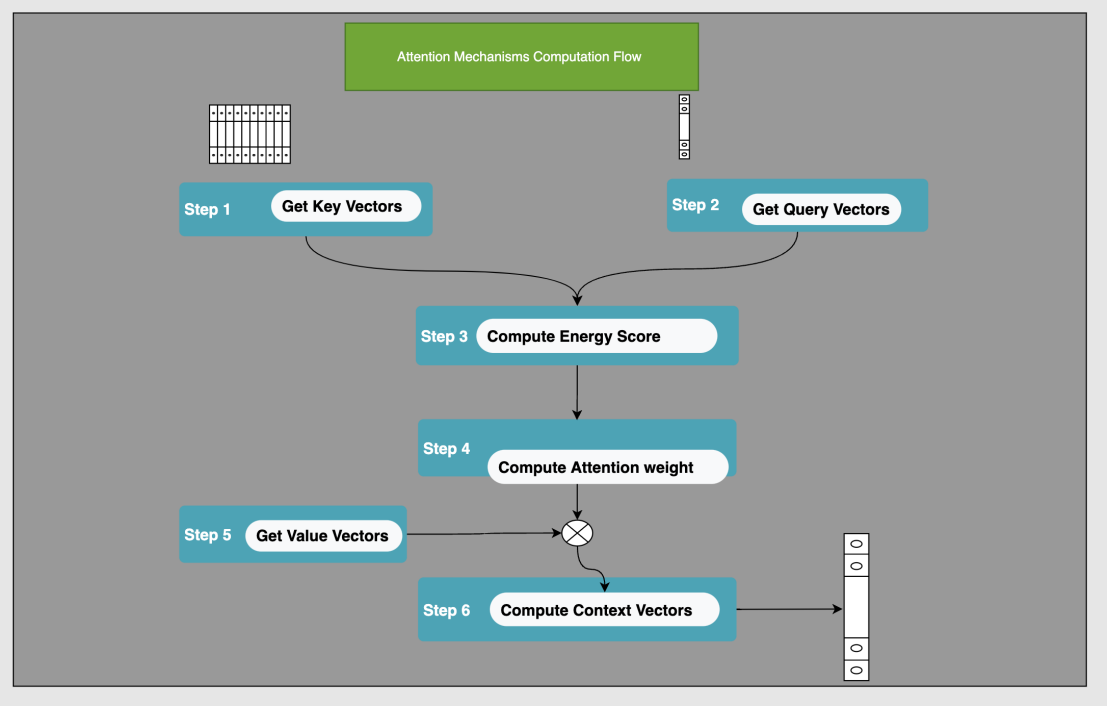
$Z=W*V$

Each element of V corresponds to one and only one element of K, and the two can be seen as different representations of the same data.

$zi=wi*vi,$

$C=\sum dki=1(zi)$

# How to compute attention?



## 1. Get KEY

```
#Input
sentences = ['food', 'is', 'delicious', 'but', 'price', 'is', 'high']

#Dimensionality reduction for easy visualization.
pca = PCA(n_components=5)

# Key sequence
KEY = [glove_vectors[i].tolist() for i in sentences]
KEY = pca.fit_transform(KEY)
```

## 2. Get Query

```
#Query text
target = [ 'cost']

#Query embedding and dimensionality reduction
QUERY = [glove_vectors[i].tolist() for i in target]
QUERY = pca.transform(QUERY)
```

## 3. Compute Energy Function.

Here, Energy Function is DotProduct.

```
def compute_energy_function(vect, mat):
  # calculate energy/attention fuction
  return np.matmul( mat, np.transpose(vect),)

ENERGY_SCORE = compute_energy_function(QUERY, KEY)
ENERGY_SCORE
```

## 4. Compute Attention weights

```
def softmax(x):
    # Compute attention weights
    x = np.array(x, dtype=np.float64)
    e_x = np.exp(x)
    return e_x / e_x.sum(axis=0)

ATTN_WEIGHTS = softmax(ENERGY_SCORE)

attn_df = pd.DataFrame(ATTN_WEIGHTS, index = sentences, columns = target)
```

## 5. VALUE

```
VALUE = np.matrix(KEY)
VALUE[:,1].reshape(1,-1).tolist()
ATTN_WEIGHTS[:, 0][0]
```

## 6. Context Vectors

```
def apply_attention_scores(attention_weights, annotations):
 # Multiple the annotations by their attention weights
 return  [  annotations[:, i] * attention_weights[i] for i in range(len(attention_we

applied_attention_1 = apply_attention_scores(ATTN_WEIGHTS, np.transpose(VALUE))
def calculate_attention_vector(applied_attention):
 return np.sum(applied_attention, axis=0)

#applied_attention = np.array([[applied_attention_1]])
attention_vector = calculate_attention_vector(applied_attention_1)
```

## Attention Mechanisms can be broadly categorized into four category:

1. Number of Sequences(Self Attention):

- Distinctive Attention Mechanisms
- Self Attention Attention Mechanisms

2. Number of Abstraction(Multi-Level Attention)

- Single Level Attention Mechanisms
- Multi Level Attention Mechanisms

3. Number of Position(Local Attention)

- Soft Attention Mechanisms
- Hard Attention Mechanisms

4. Number of Representation(Multi-head Attention)

- Single Head Attention Mechanisms
- Multi Head Attention Mechanisms

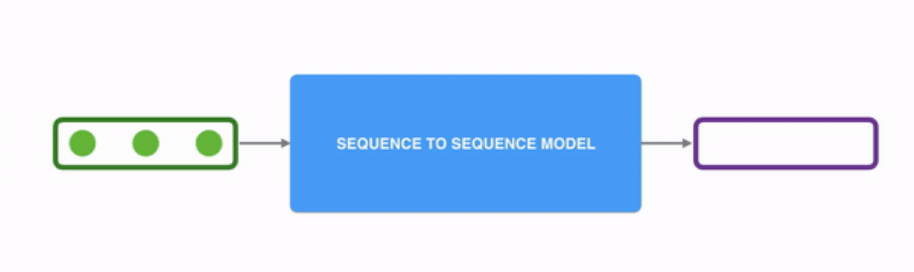
# Lecture 10: Transformers (Encoder-Decoder Architectures)

## Content

- Seq2Seq Learning algorithms
  - Machine Translation algorithms
- Encoder-Decoder Architecture
- RNN based Encoder-Decoder
- Transformer based Encoder-Decoder

## Seq2Seq Learning algorithms

These are algorithms where inputs and outputs are set of sequences. These sequences can be anything like words, characters etc.

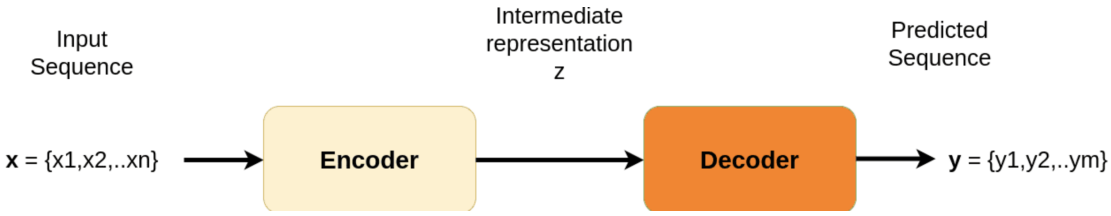



Machine translation is applying seq2seq algorithms to translate sentences from one language to another. Major algorithms for machine translation include

| Abbreviation | RBMT | SMT | NMT |
|---|---|---|---|
| Name | Rule Based Machine Translation | Statistical Machine Translation | Neural Machine Translation |
| Overview | Machine translation based on dictionary and knowledge of grammar | Machine translation based on statistical information derived from large volume of collected parallel data | Utilizes AI<br>Machine translation using deep learning technology |
| Pros | Translation speed is fast<br>Faithful to the original<br>Suitable for standard translation | Translations are relatively natural sentences, higher than the quality of RBMT<br>It is also easy to get high BLEU* scores with SMT | Can produce more natural sentences, and higher translation quality than SMT |
| Cons | Translation lacks naturalness | Translation lacks naturalness | Takes time to learn target data |

Neural Machine Translation is the state-of-the-art algorithm as of 2023

## Encoder-Decoder Architecture

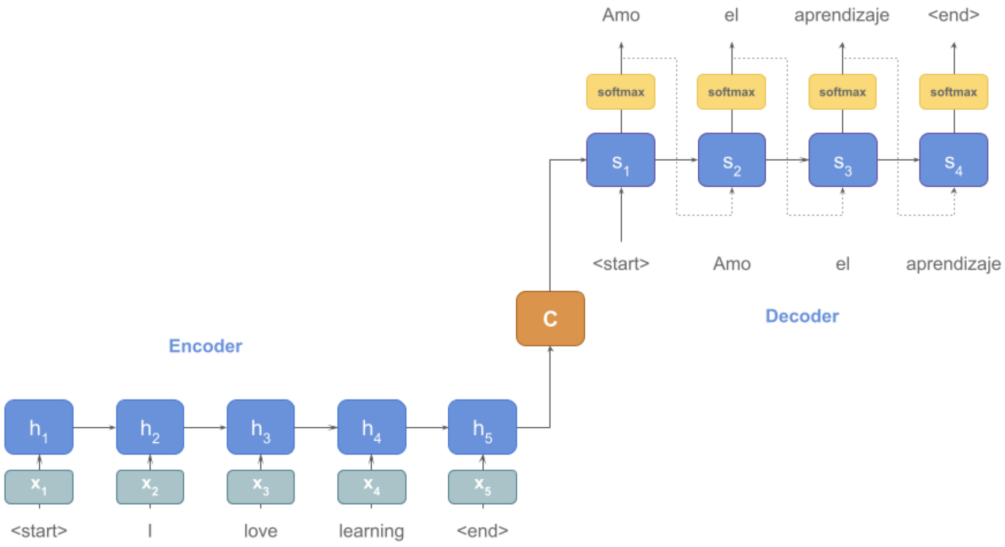



Encoder decoder architecture consists of two components

- Encoder processes input sequence to a hidden representation, which captures inner meaning of text in terms hidden vectors
- Decoder converts hidden representation to output sequence

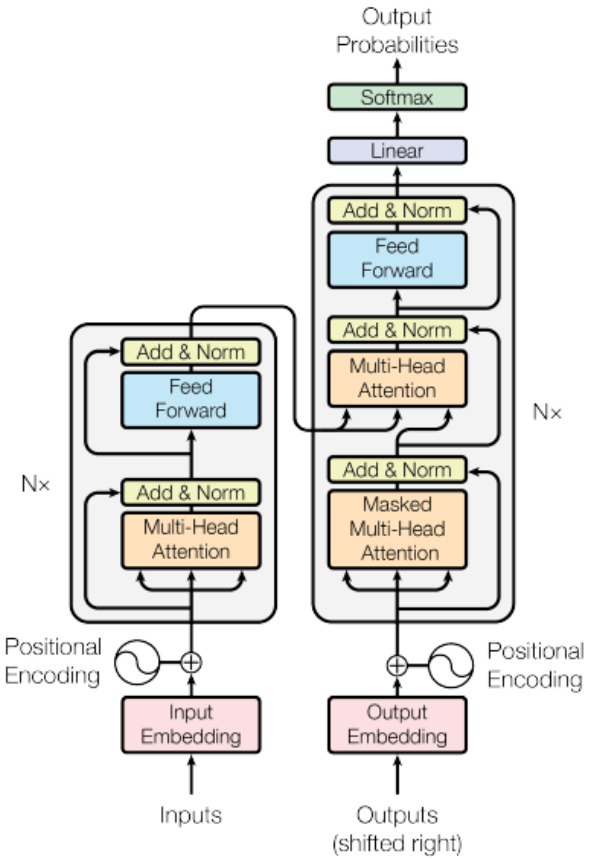
## RNN/LSTM/GRU based Encoder-Decoder

- Encoder
  - Recieves token embeddings as input
  - Processed though multiple RNN stacks
  - Last hidden state (and cell state) captures hidden representation in RNN/LSTM/GRU based encoder
- Decoder
  - Initalized with encoder's hidden representation
  - Processed through multiple RNN stacks
  - Outputs from final RNN layer are used to calculate loss / logits




- Problems with RNN/LSTM/GRU based NMT
  - Unable to capture long term dependencies
  - Architeture is not parallelizable

## Transformers based Encoder-Decoder



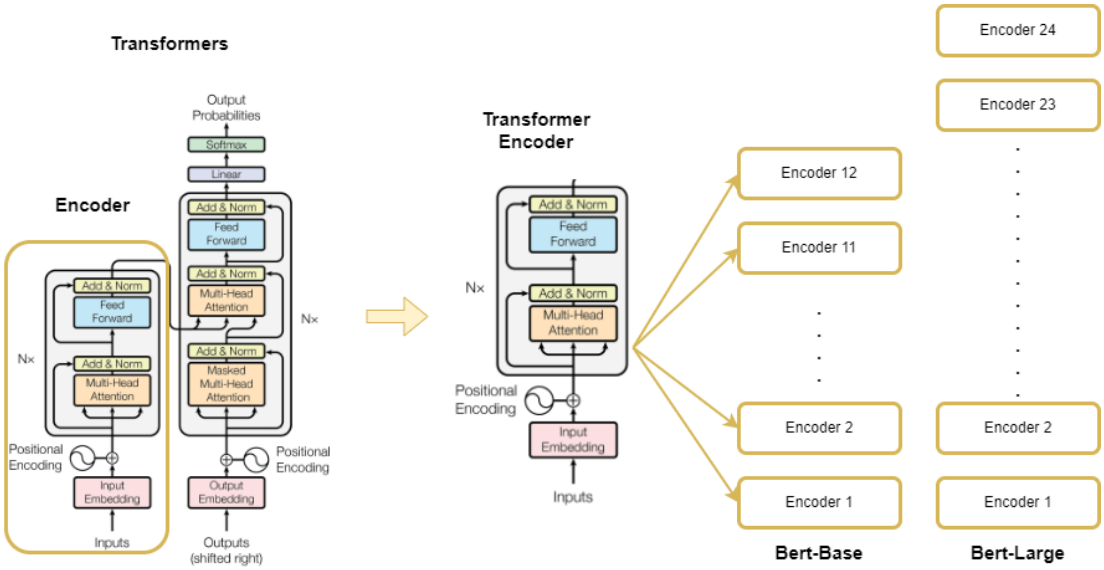

- Encoder block
  - Embeddings of tokenized inputs are added to positional embeddings to serve as inputs to encoder block
  - Each encoder block contains a Multi-Head Attention networks and a Feedforward network. Outputs of these networks are added to it's inputs and then normalized.
  - Each head of Multi-Head Attention computes attention pattern between projected queries and keys. This pattern is matrix multiplied by values. Multiplied values from each head are concatenated and projected to get outputs.

    - Feed forward network is a simple linear projection to a hidden dimension and another projection to embed dimension.
- Decoder block
    - Embeddings of tokenized inputs are added to positional embeddings to serve as inputs to decoder block too.
        - During training, inputs to a decoder block are outputs shifted to the right.
        - During inference, inputs to a decoder block is usually just a start token
    - Each decoder block contains 4 components: Masked multi head attention, feedforward 1, multihead attention and feedforward 2.
    - In masked multi-head attention, upper triangular mask is applied to attention pattern before matmul by values. Other components and flow is inputs is similar to that of encoder.
    - Finally, outputs are linear projected to vocab dimension and logits are obtained. Categorical cross entropy is typically used as loss function.

# Lecture 11: Bert

# Bert

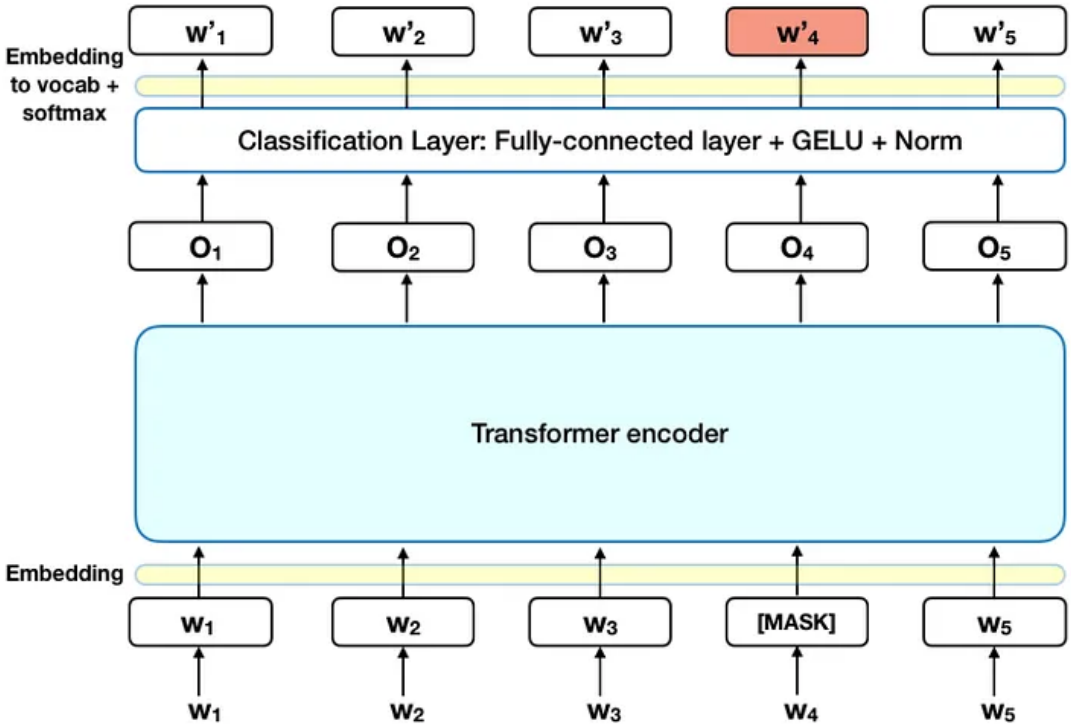
- BERT stands for BiDirectional Encoder Representation from Transformers.
- BERT uses only the Encoder portion of the transformer's architecture which is the reason it's called **Encoder Representation from transformers.**




Bert uses two training mechanisms namely **Masked Language Modeling (MLM)** and **Next Sentence Prediction (NSP)** to overcome the dependency challenge.

## Masked Language Modeling (MLM)

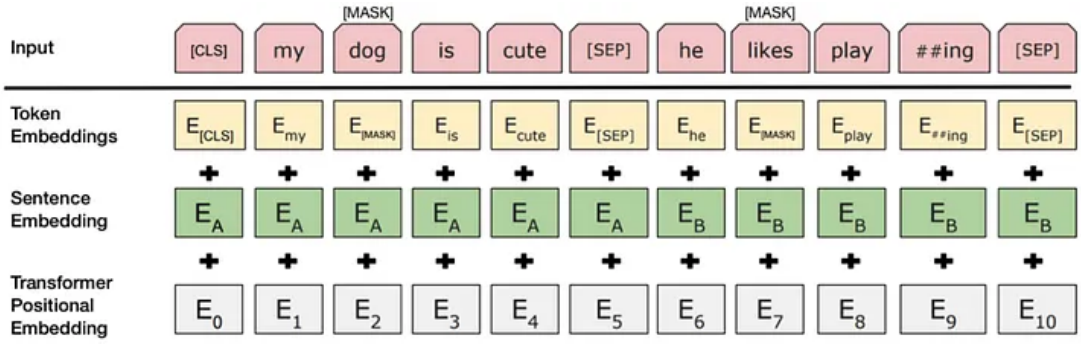
Before feeding the input vector into the encoder, 15% of the words in the sequence will be masked with a [MASK] token. The goal of MLM is to predict the masked word with respect to all other
words in the sentence.




Encoded output of the Transformer encoder is sent to a fully connected classification layer. The results of the classifier would then be multiplied by an embedding matrix to convert to the vocabulary

## Next Sentence Prediction (NSP)

While training, the model would receive pairs of sentences as inputs.The model eventually learns it and predicts whether the second sentence in the pair is actually the consecutive sentence or not.




We can understand, a[CLS] token at the beginning and a [SEP] token at the end of a sentence are added.

To predict whether the second sentence is connected to the first sentence, the entire input sequence would be sent to a Transformer encoder and then the output of the [CLS] token is transformed into a 2×1 shaped vector, using a simple classification layer.

## There are 4 variants of BERT based on the number of encoder blocks and Attention heads

| Variants | Encoder Block | Attention heads | Hidden size | Case Sensitive | Parameters |
|---|---|---|---|---|---|
| Bert-Base-uncased | 12 | 12 | 768 | No | 110M |
| Bert-Base-cased | 12 | 12 | 768 | Yes | 110M |
| Bert-Large-uncased | 24 | 16 | 1024 | No | 340M |
| Bert-Large-cased | 24 | 16 | 1024 | Yes | 340M |

## More architectures based on Transformers

| | BERT | RoBERT | DistilBERT | XLNet |
|---|---|---|---|---|
| **Size (millions)** | **Base**: 110<br>**Large**: 340 | **Base**: 110<br>**Large**: 340 | **Base:** 66 | **Base**: ~110<br>**Large**: ~340 |
| **Training Time** | **Base**: 8 x V100 x 12 days*<br>**Large:** 64 TPU Chips x 4 days (or 280 x V100 x 1 days*) | **Large:** 1024 x V100 x 1 day; 4-5 times more than BERT. | **Base**: 8 x V100 x 3.5 days; 4 times less than BERT. | **Large:** 512 TPU Chips x 2.5 days; 5 times more than BERT. |
| **Performance** | Outperforms state-of-the-art in Oct 2018 | 2-20% improvement over BERT | 5% degradation from BERT | 2-15% improvement over BERT |
| **Data** | 16 GB BERT data (Books Corpus + Wikipedia).<br>3.3 Billion words. | 160 GB (16 GB BERT data + 144 GB additional) | 16 GB BERT data.<br>3.3 Billion words. | **Base**: 16 GB BERT data<br>**Large**: 113 GB (16 GB BERT data + 97 GB additional).<br>33 Billion words. |
| **Method** | BERT (Bidirectional Transformer with MLM and NSP) | BERT without NSP** | BERT Distillation | Bidirectional Transformer with Permutation based modeling |